CS251: Stack overflowers
Project: **Personal Budgeting Software**

Software Design Specification

Due date: April 2025

Contents

Software Design Specification

System Models

Architecture Diagram

We chose a **Three-Tier Layered + Microservices Architecture** because it provides:

1. **Separation of Concerns:** The UI is separated from business logic and data management.
2. **Scalability:** Each service (e.g., Transaction, Reporting) can be scaled independently based on demand.
3. **Maintainability and Flexibility:** New services can be added or updated without affecting the rest of the system.

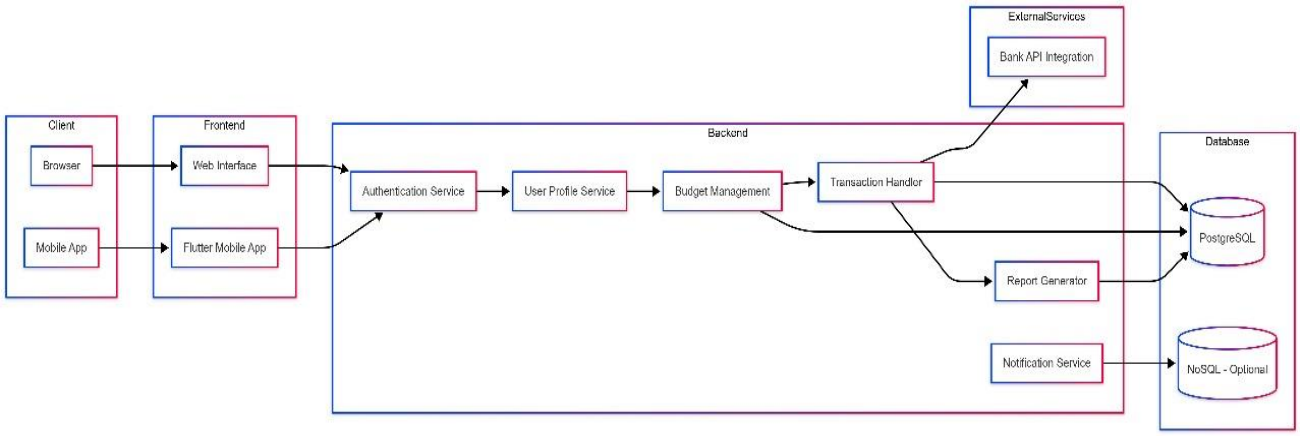
• **System Components:**

1. **Presentation Layer:**
    - Web Interface (React/Vue)
    - Mobile App (Flutter)
2. **Service Layer:**
    - Authentication, User Profile, Budget Management, Transaction Handler, Report Generator, Notification Handler, Bank API Integration
3. **Data Layer:**
    - PostgreSQL (for structured transaction data)
    - NoSQL (e.g., MongoDB/Redis) for flexible notification storage and logs

CS251: Stack overflowers

Project: **Personal Budgeting Software**

Software Design Specification



Software Design Specification

II. Class Diagram(s)

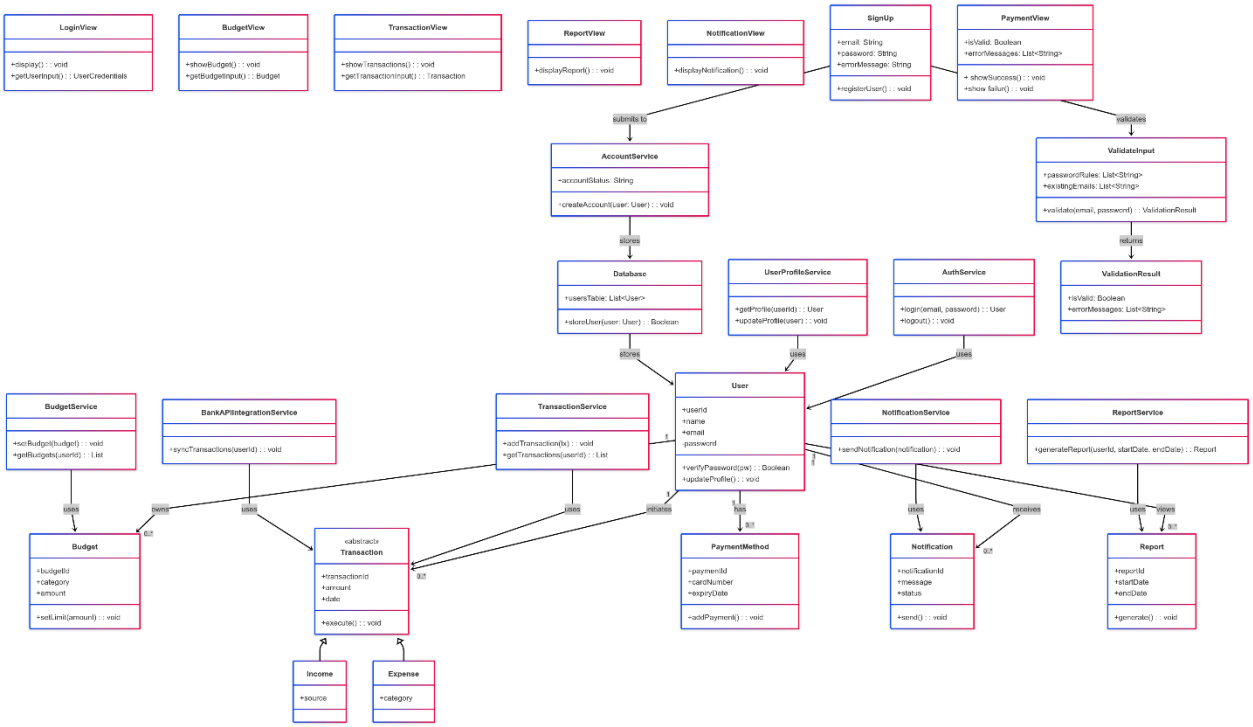
The class diagram represents the structure of the system, focusing on **Entity Classes**, **Boundary Classes**, and **Control Classes**.

Entity Classes: These represent the core domain objects such as User, Budget, Transaction (with subclasses Income and Expense), PaymentMethod, Notification, and Report. Each class includes relevant attributes and methods, such as validation methods and operations to handle the core functionality of the system.

Boundary Classes: Represent the user interface components interacting with the users, such as LoginView, BudgetView, and TransactionView. These classes facilitate the flow of data between the UI and the control classes.

Control Classes: Represent the services that handle business logic and control data flow. These include AuthService, UserProfileService, BudgetService, and others. Each service interacts with the relevant entity classes to perform operations like authentication, transaction management, and budget control.

Relationships: The diagram also shows relationships between the classes, such as the User class owning multiple Budget instances or initiating multiple Transaction instances. The <<uses>> stereotype indicates that service classes use the relevant entity classes to perform their functions.



Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1	`User` <<entity>>	Represents a system user. Stores user credentials & profile data, verifies passwords and updates profile.
2	`Budget` <<entity>>	Encapsulates a user's budget category and limit. Allows setting and retrieving budget constraints.
3	`Transaction` <<entity>>	Abstract base for all financial transactions. Defines common attributes (`amount`, `date`) and `execute()`.
4	`Income` <<entity>>	Subclass of `Transaction` for incoming funds. Adds `source` field to record income origin.
5	`Expense` <<entity>>	Subclass of `Transaction` for spending. Adds `category` field to classify expenses.
6	`PaymentMethod` <<entity>>	Models a user's payment instrument (e.g. credit card). Manages adding and storing card details.
7	`Notification` <<entity>>	Represents an alert/message to a user. Stores message text and status, and provides `send()` functionality.
8	`Report` <<entity>>	Encapsulates report parameters and generation. Holds date range and invokes `generate()` to build the report.
9	`LoginView` <<boundary>>	UI boundary for authentication. Renders login form and captures credentials via `getUserInput()`.
10	`BudgetView` <<boundary>>	UI boundary for budgeting. Displays existing budgets and captures new budget data from the user.
11	`TransactionView` <<boundary>>	UI boundary for transactions. Lists transactions and captures new transaction details from the user.
12	`ReportView` <<boundary>>	UI boundary for reporting. Displays generated financial reports to the user.

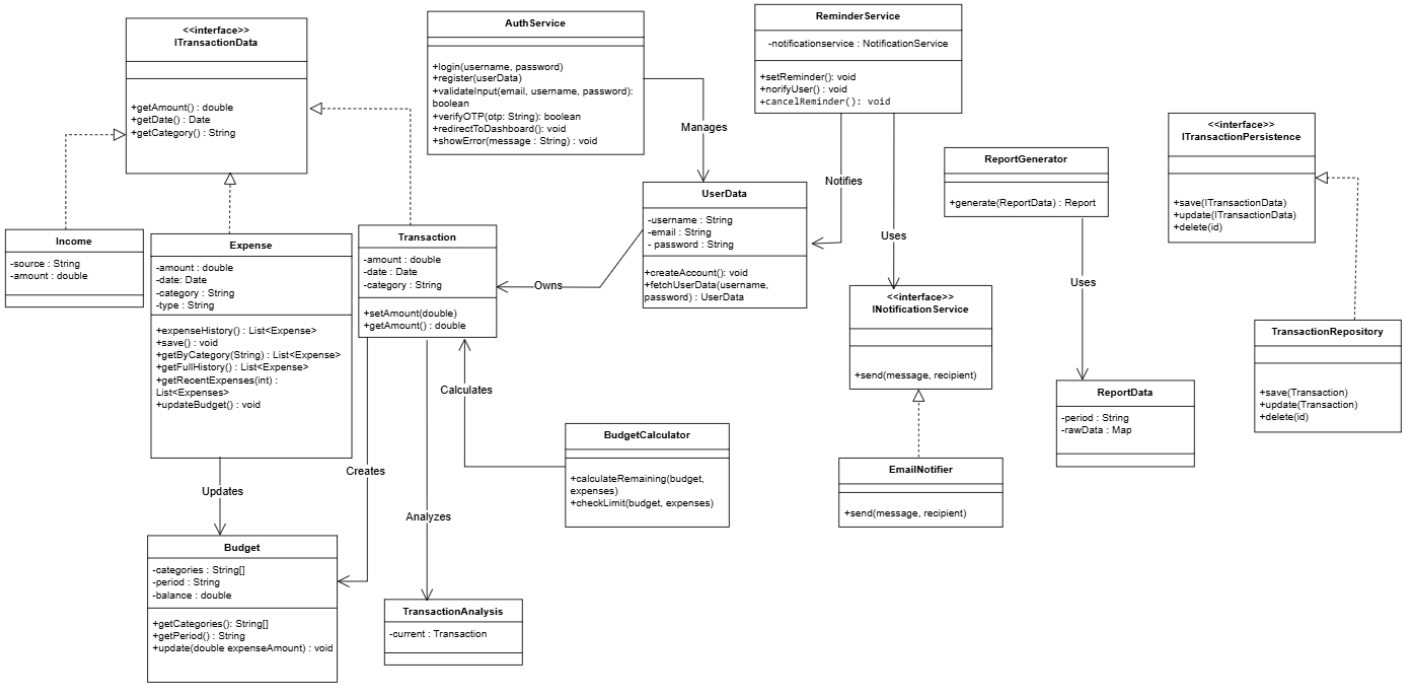
Software Design Specification

13	`NotificationView` <<boundary>>	UI boundary for notifications. Presents alert messages and status to the user.
14	`AuthService` <<control>>	Business logic for authentication. Performs `login()` and `logout()`, interacts with `User` entity.
15	`UserProfileService` <<control>>	Manages user profile operations. Retrieves and updates `User` data (`getProfile()`, `updateProfile()`).
16	`BudgetService` <<control>>	Handles budgeting logic. Creates and retrieves budgets for a user (`setBudget()`, `getBudgets()`).
17	`TransactionService` <<control>>	Orchestrates transaction operations. Adds new transactions and fetches history (`addTransaction()`, `getTransactions()`).
18	`ReportService` <<control>>	Coordinates report generation. Invokes `Report.generate()` over a date range and returns the result.
19	`NotificationService` <<control>>	Sends out notifications. Accepts `Notification` objects and invokes their `send()` method.
20	`BankAPIIntegrationService` <<control>>	Synchronizes with external bank APIs. Pulls or pushes transaction data (`syncTransactions()`).
21	`SignUp` <<entity>>	Manages the user registration process, collecting the user's email and password, and handling any errors that occur during registration
22	`ValidateInput` <<control>>	Validates the inputs entered during the sign-up process by checking password rules and whether the email is already registered
23	`AccountService` <<control>>	Handles the backend logic of creating user accounts, including setting the account status and initializing user-specific settings.

Software Design Specification

24	`Database` <<entity>>	Simulates a storage system for users, holding the list of all registered users and providing methods to store new users.
25	`ValidationResult` <<boundary>>	Represents the outcome of input validation, indicating whether the input was valid and containing a list of error messages if not
25	`PaymentView` <<entity>>	Represent the validation of the input and shows the success of failure messages

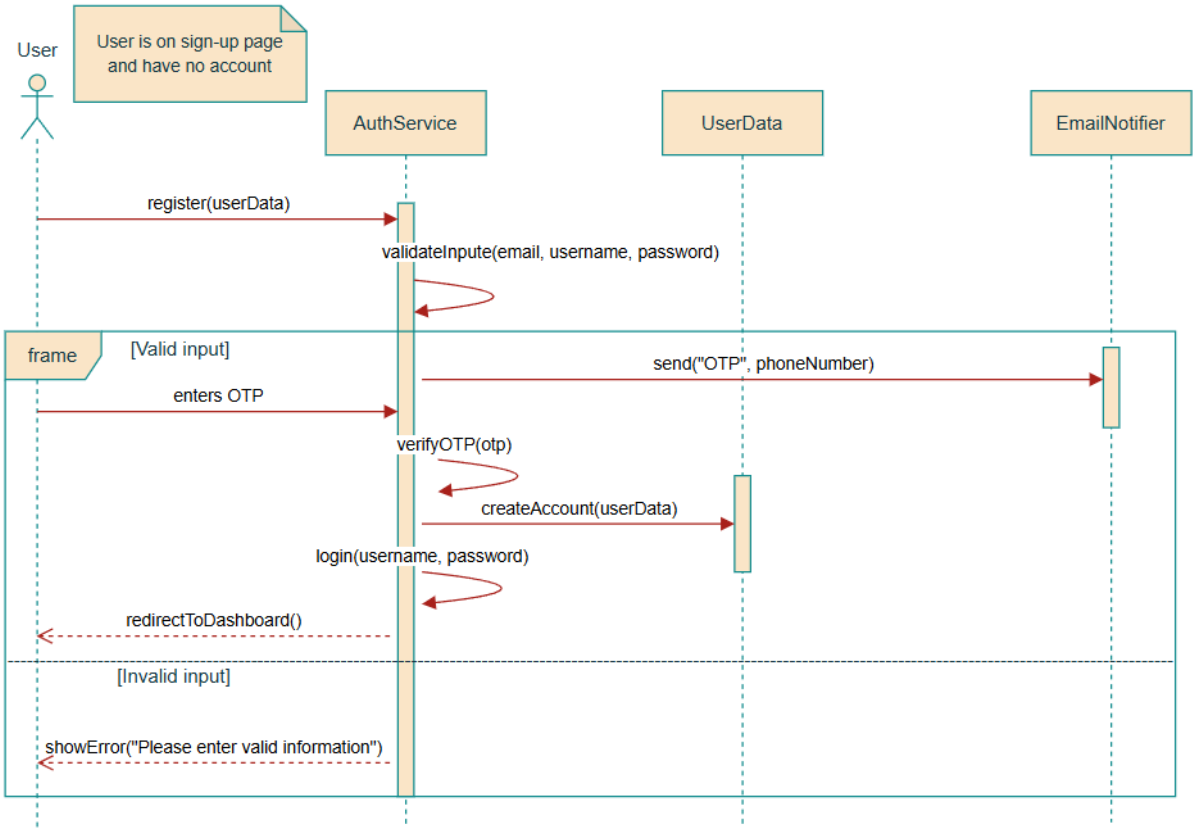
ClassDiagram II



Software Design Specification

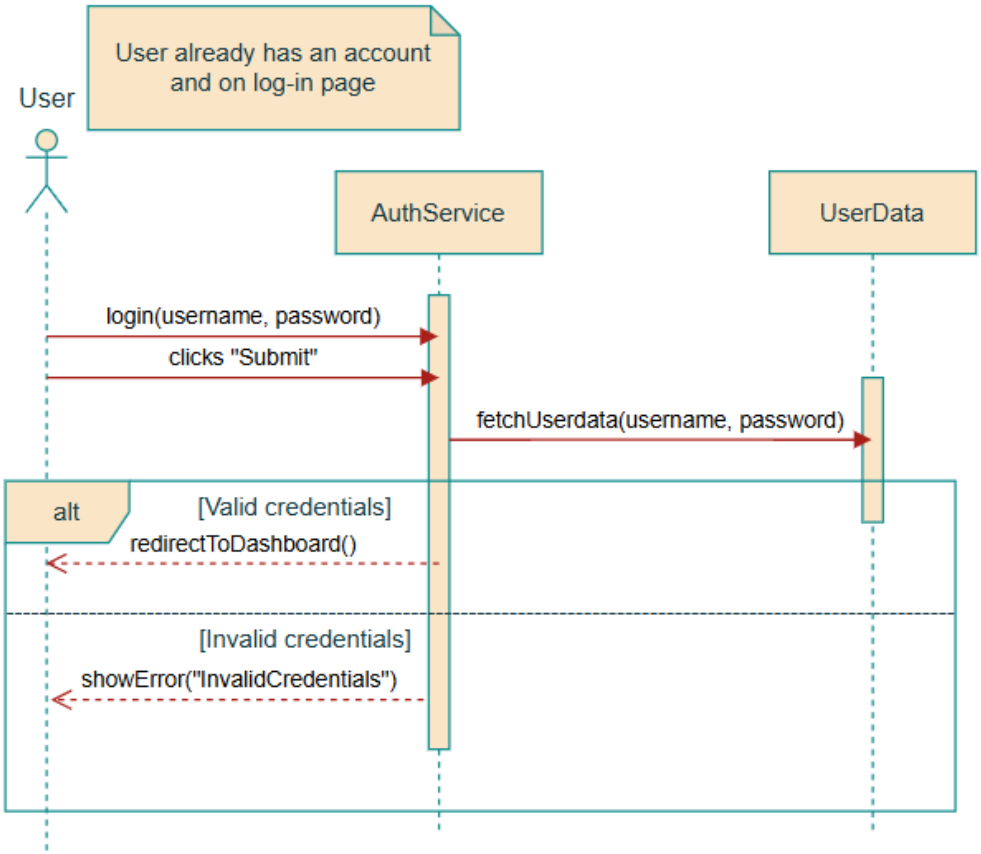
IV. Sequence diagrams

User story #1: Sign up



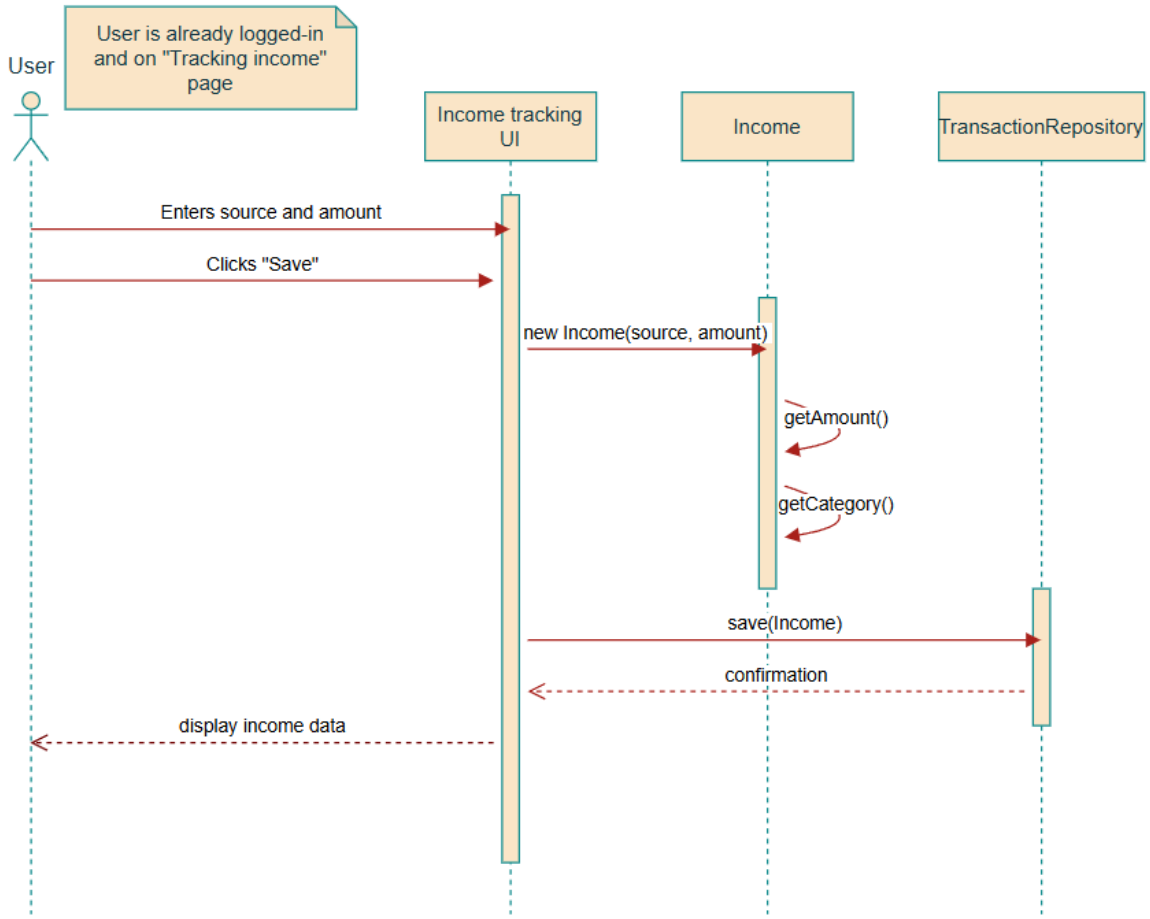
Software Design Specification

User story #2: Log-in



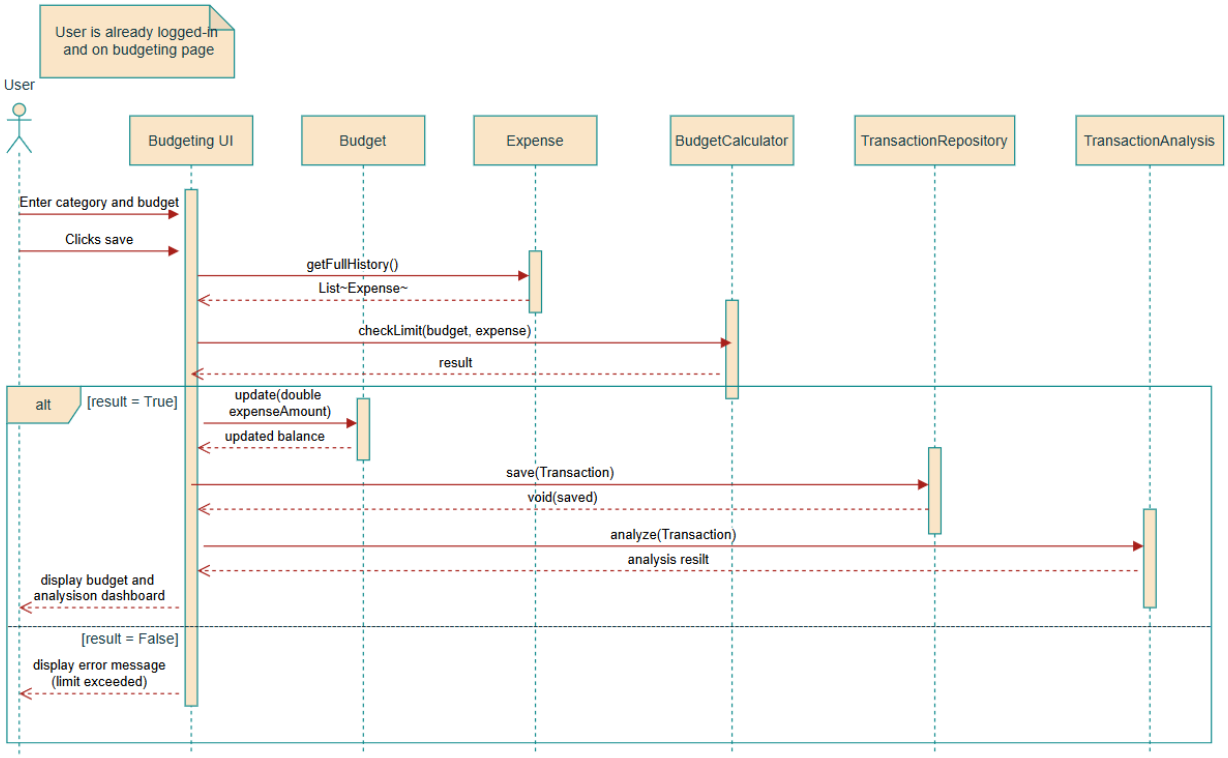
Software Design Specification

User story #3: Tracking income



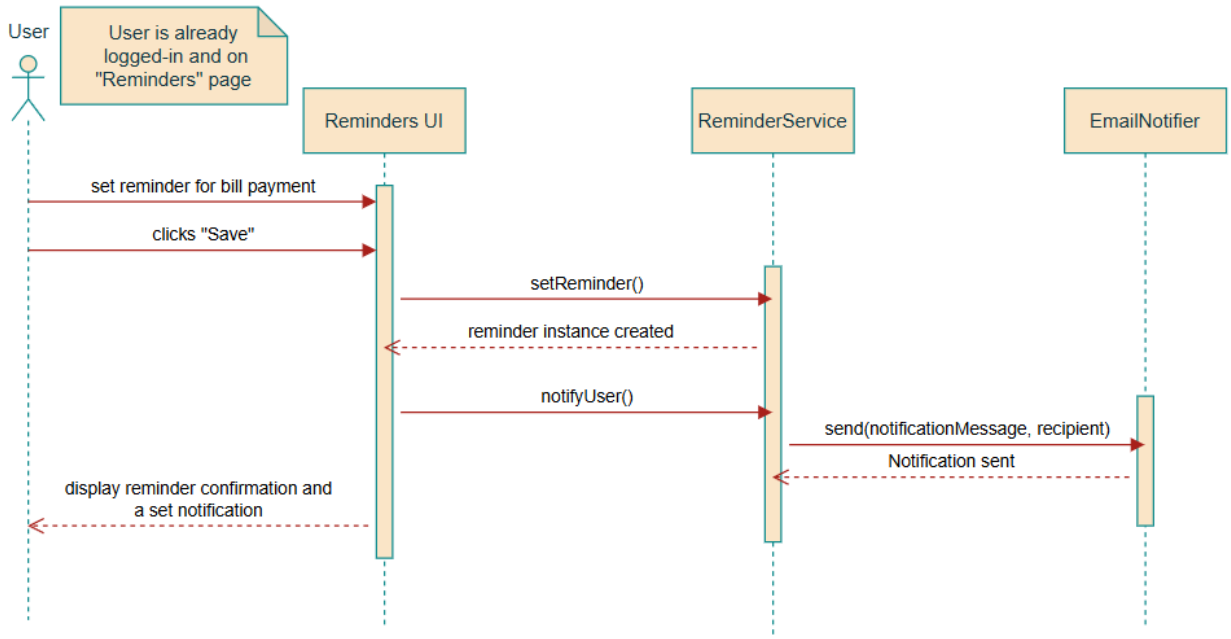
Software Design Specification

User story #4: Budgeting & Analysis



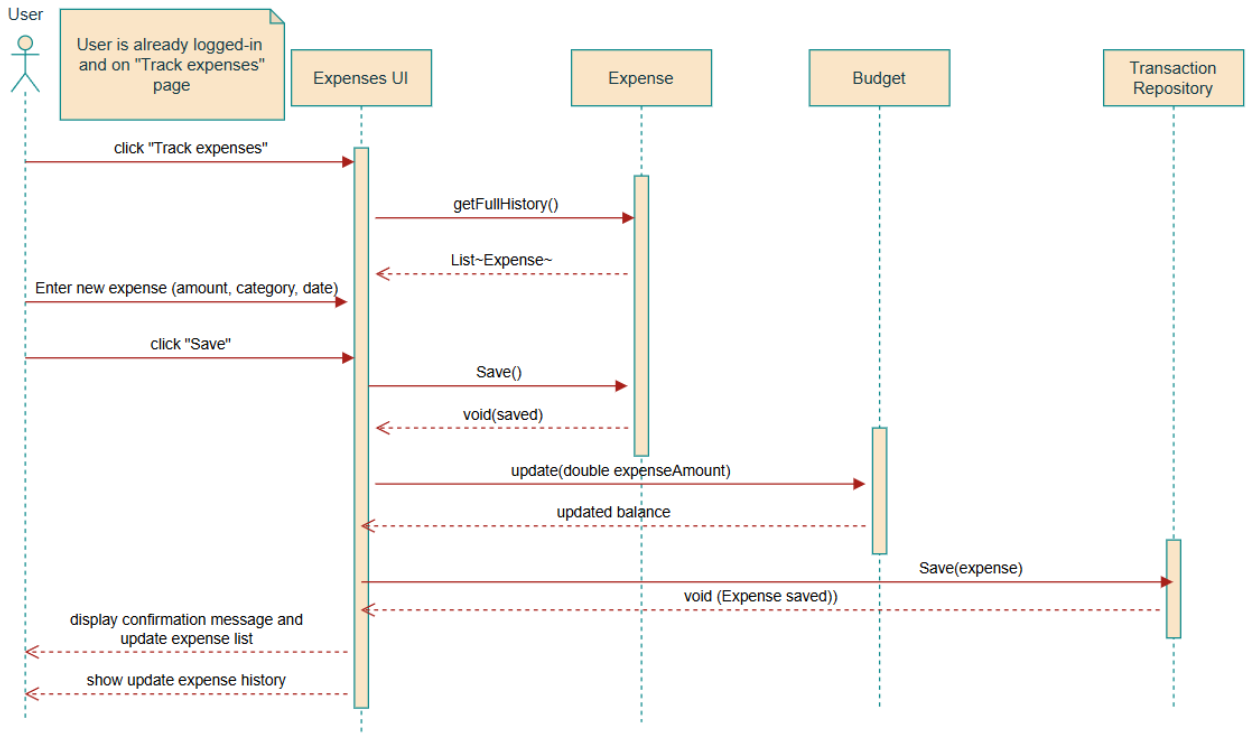
Software Design Specification

User story #5: Reminders



Software Design Specification

User story #6: Expense Tracking



Class - Sequence Usage Table

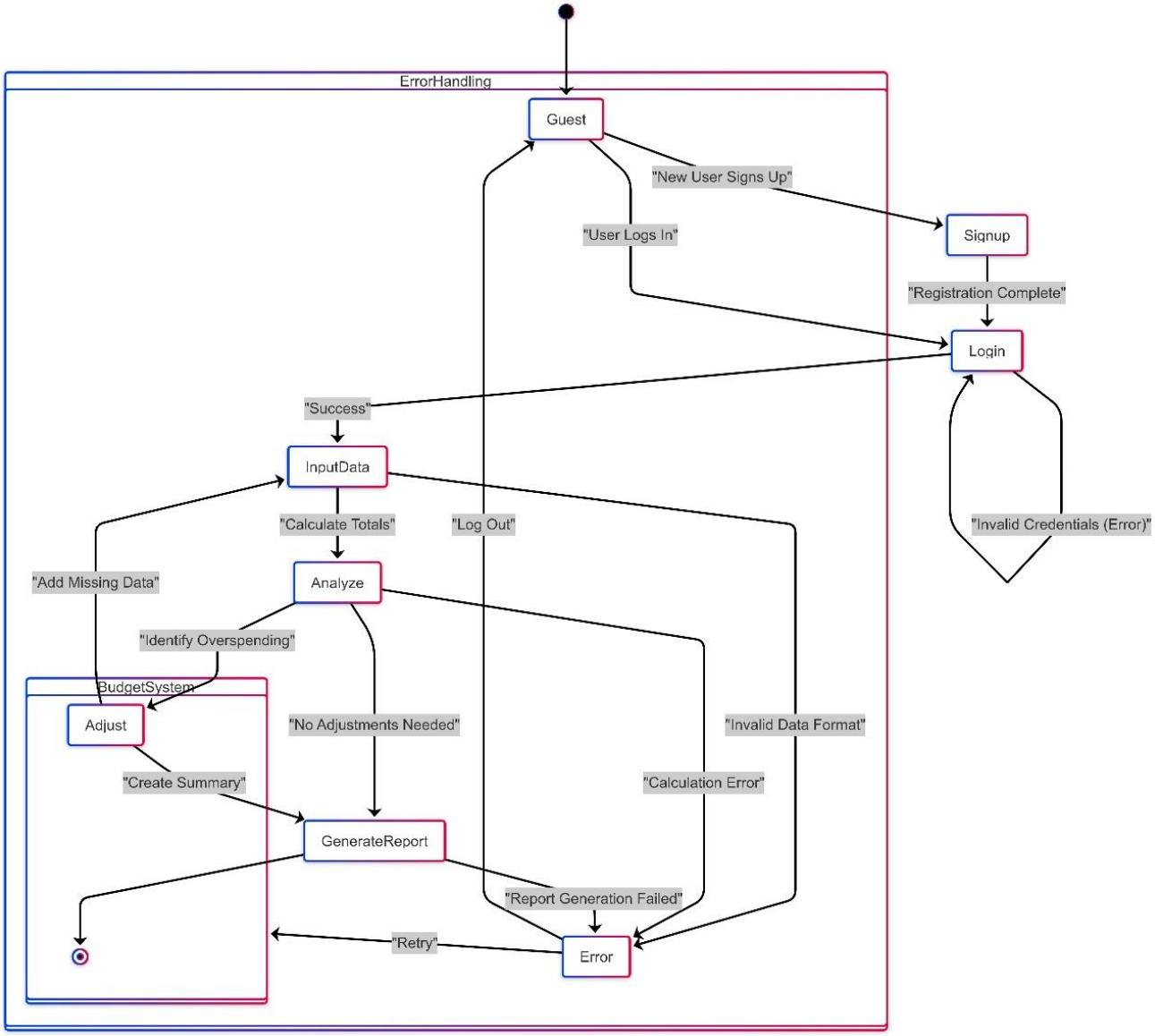
Sequence Diagram	Classes Used	All Methods Used
1. Sign-up	• Authservice • UserData • EmailNotifier	• register(userData) • validateInput() • send() • verifyOTP() • CreateAccount(userData) • Login(username, password) • redirectToDashboard()
2. Log-in	• Authservice • UserData	• showError() • Login(username, password) • fetchUserData(username, password) • redirectToDashboard()

Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
3. Tracking income	• Income • TransactionRepository	• Income() • Save(0 • getAmount() • getCategory()
4. Budgeting & analysis	• Budget • Expense • BudgetCalculator • TransactionRepository • TransactionAnalysis	• getFullHistory() • checkLimit(budget, expense) • update(expenseAmount) • save() • analyze()
5. Reminders	• ReminderService • EmailNotifier	• setReminder() • notifyUser() • send()
6. Tracking expense	• Expense • Budget • TransactionRepository	• getFullhistory() • save() • update(expenseAmount)

Software Design Specification

V. State Diagram



Software Design Specification

VI. SOLID Principles

1. Single Responsibility Principle (SRP):

- **SRP** states that a class should have only one reason to change, meaning each class should be responsible for a single part of the functionality.
- **In code (Implementation)**:
    - **Transaction**, **Income**, and **Expense** classes seem focused on different transaction types, each handling specific aspects of transaction data.
    - **TransactionRepository** handles persistence, **Budget** handles the budget calculations, and **ReportGenerator** handles report generation.
    - **ReminderService** handles the notification reminder, which can be seen as a separate responsibility from transaction handling or user data.

2. Open/Closed Principle (OCP):

- **OCP** states that classes should be open for extension but closed for modification. You should be able to extend the behavior of a class without changing its existing code.
- **In code (Implementation)**:
    - **Interfaces** like ITransactionData, ITransactionPersistence, and INotificationService make it easy to extend functionality without modifying existing code. For example, adding new transaction types (such as Income or Expense) or adding new persistence mechanisms (like a TransactionRepository implementation using a different database) can be done by simply implementing the interfaces.
    - **Transaction** class is easily extendable by adding more fields or methods, while **TransactionAnalysis** can be extended with different types of analysis without modifying the Transaction class.

Software Design Specification

3. **Liskov Substitution Principle (LSP):**

- **LSP** states that objects of a superclass should be replaceable with objects of a subclass without affecting the functionality of the program.
- **In code (Implementation)**:
    - **Transaction**, **Income**, and **Expense** all implement the ITransactionData interface. This ensures that they can be substituted for each other in contexts that expect ITransactionData objects (such as TransactionRepository or BudgetCalculator), without altering the expected behavior.
    - The fact that both **Income** and **Expense** can inherit from ITransactionData ensures that their objects can be used interchangeably in code that depends on ITransactionData, maintaining the integrity of the LSP.

4. **Interface Segregation Principle (ISP):**

- **ISP** states that clients should not be forced to depend on interfaces they do not use. This encourages small, specific interfaces.
- **In code (Implementation)**:
    - **ITransactionData** and **ITransactionPersistence** are well-defined interfaces, each containing only the relevant methods for their respective responsibilities.
    - **INotificationService** is a separate interface for notification functionality, so clients depending on notifications won't need to know about transaction persistence or data models.
    - Each class that implements these interfaces (like **TransactionRepository**, **EmailNotifier**) only implements the methods they need, and thus, does not implement unnecessary methods.

Software Design Specification

5. **Dependency Inversion Principle (DIP):**

- **DIP** states that high-level modules should not depend on low-level modules; both should depend on abstractions. Also, abstractions should not depend on details; details should depend on abstractions.
- **In code (Implementation)**:
    - The high-level modules (like **ReminderService** or **TransactionAnalysis**) depend on abstractions like INotificationService and ITransactionData, rather than concrete implementations (like EmailNotifier or Transaction).
    - This reduces the coupling between modules and allows easier extension and maintenance.

VII. Design Patterns

1. Strategy Pattern

Intent: Define a family of algorithms (strategies), encapsulate each one, and make them interchangeable.

Where it appears:

- Interfaces such as ITransactionData, ITransactionPersistence, and INotificationService define interchangeable behaviors.
- Implementation:
    - TransactionRepository implements ITransactionPersistence, allowing different strategies for saving, updating, or deleting transaction data.
    - EmailNotifier implements INotificationService, which could be swapped out with other notifiers (e.g., SMS, push notification) without changing ReminderService.

Software Design Specification

Explanation:
Using interfaces for transaction data and services lets you switch implementations at runtime or configure different strategies depending on context.

2. Dependency Injection (a technique often used with the Strategy Pattern)

Intent: Reduce hard dependencies between components by injecting required dependencies from outside.

Implementation:

- ReminderService depends on INotificationService rather than a concrete EmailNotifier.

Explanation:
This allows the use of different notification mechanisms by simply injecting another class that implements the INotificationService interface.

3. Repository Pattern

Intent: Encapsulate data access logic and provide a clean interface for the domain.

Implementation:

- TransactionRepository acts as a **repository** that hides the actual data storage mechanism and provides an API (save, update, delete) to manage Transaction entities.

Explanation:
The repository provides a level of abstraction between the domain (e.g., Expense, Income) and the data source, improving testability and separation of concerns.

Tools

- Draw.io → Sequence Diagrams
- Mermaid Chart & Draw.io→ Class Diagram

CS251: Stack overflowers
Project: **Personal Budgeting Software**

Software Design Specification

Ownership Report

Student	Owners
Ahmed Mohamed Mahmoud	Part of Class Description & Class-Sequence Usage Table & design patterns
Mariam Ali Abd El Kereem	Class Diagram & Part of Class Description & Architecture Diagram & State Diagram
Nagham Sabry Ahmed	Sequence Diagrams & Organizing the File & SOLID principles